

Yazılım Mimarileri ve Tasarım Yöntemleri Dersi

Dönem Projesi Teknik Raporu

QMOOD Tabanlı Yazılım Kalitesi Analizi ve LLM Destekli Değerlendirme

İncelenen Proje: picocli

Hazırlayan	[Ad Soyad / Grup Üyeleri]
Ders	Yazılım Mimarileri ve Tasarım Yöntemleri
Analiz Edilen Sürüm Sayısı	10
Kullanılan Ana Araçlar	CK Tool, Python, Pandas, Matplotlib
Rapor Durumu	QMOOD analizi tamamlandı; LLM değerlendirme bölümü için prompt ve rubrik taslağı eklendi

Özet

Bu çalışmada açık kaynak kodlu Java tabanlı picocli projesinin 10 farklı sürümü üzerinde nesneye dayalı yazılım metrikleri çıkarılmış, bu metrikler QMOOD kalite modeli kapsamında tasarım özelliklerine ve kalite niteliklerine dönüştürülmüştür. İncelenen sürümler v4.0.4 ile v4.7.7 arasındadır. Analiz sürecinde CK Tool ile class-level ve method-level metrikler çıkarılmış; ardından Python kullanılarak sürüm bazlı özet metrikler, normalize QMOOD skorları ve grafikler üretilmiştir.

Sonuçlara göre picocli projesinin sınıf sayısı 192'den 219'a, metot sayısı 1517'den 1848'e yükselmiştir. Toplam kalite indeksi (TQI) v4.0.4 sürümünde 4.0100 iken v4.7.7 sürümünde 4.1015 olarak hesaplanmıştır. Bu yaklaşık %2.28 düzeyinde bir artışa karşılık gelmektedir. Reusability, flexibility, functionality ve effectiveness nitelikleri artarken understandability değerinde düşüş gözlenmiştir. Bu durum yazılımın işlevsel olarak genişlediğini, ancak büyümeye bağlı karmaşıklık ve cohesion risklerinin izlenmesi gerektiğini göstermektedir.

İçindekiler

1. Giriş
2. Literatür Özeti
3. Yöntem
4. Analiz Süreci
5. Sonuçlar
6. Tartışma
- 6.5. LLM Çıktılarının Karşılaştırmalı Analizi
- 6.6. Prompt 2 ile Yapılandırılmış LLM Değerlendirmesi
- 6.7. v4.7.7 Sınıf Düzeyi Risk Analizi
7. Sonuç ve Gelecek Çalışmalar
- Ek A. LLM Promptları ve Değerlendirme Rubriği
12. Kaynakça

1. Giriş

Yazılım sistemleri zaman içinde yeni gereksinimler, hata düzeltmeleri ve tasarım kararları nedeniyle evrilir. Bu evrim sürecinde yazılımın işlevselliği artarken tasarım karmaşıklığı, sınıflar arası bağımlılık ve bakım maliyeti de artabilir. Bu nedenle yazılım mimarisi ve tasarım kalitesi yalnızca sezgisel olarak değil, ölçülebilir metrikler üzerinden de değerlendirilmelidir.

Bu raporun amacı, picocli adlı açık kaynaklı Java projesinin seçilen 10 sürümünde nesneye dayalı tasarım metriklerini incelemek, QMOOD kalite modeline göre kalite niteliklerini hesaplamak ve yazılımın sürümler arasındaki kalite evrimini yorumlamaktır. Çalışma ayrıca LLM destekli kalite değerlendirmesi için kullanılacak prompt yapısını ve değerlendirme rubriğini de içermektedir.

2. Literatür Özeti

2.1. Yazılım Metrikleri

Yazılım metrikleri, yazılım ürünlerinin ölçülebilen alt düzey niteliklerini ifade eder. Bu çalışmada odak, doğrudan kaynak koddan elde edilebilen statik tasarım metrikleridir. Statik metrikler program çalıştırılmadan elde edilir; örneğin sınıf sayısı, metod sayısı, bağımlılık ve kalıtım derinliği gibi ölçüler bu kapsamdadır.

2.2. CK Metrik Kümesi

Chidamber ve Kemerer tarafından önerilen CK metrik kümesi, nesneye dayalı yazılımlarda özellikle karmaşıklık, kalıtım, coupling ve cohesion gibi tasarım özelliklerinin ölçülmesinde kullanılan temel metrik kümelerinden biridir. Bu çalışmada CK Tool tarafından üretilen CBO, WMC, DIT, RFC ve LCOM gibi metriklerden yararlanılmıştır.

2.3. QMOOD Kalite Modeli

QMOOD (Quality Model for Object-Oriented Design), nesneye dayalı tasarım kalitesini hiyerarşik bir modelle değerlendirir. Modelde L4 düzeyinde tasarım bileşenleri, L3 düzeyinde tasarım metrikleri, L2 düzeyinde nesneye dayalı tasarım özellikleri ve L1 düzeyinde tasarım kalite nitelikleri yer alır. Bu yapı, düşük düzeyli metrikleri üst düzey kalite nitelikleriyle ilişkilendirmeyi amaçlar.

QMOOD kapsamında kullanılan kalite nitelikleri reusability, flexibility, understandability, functionality, extendibility ve effectiveness olarak ele alınmıştır. Bu nitelikler, normalize edilmiş tasarım özelliklerinin ağırlıklı toplamaları ile hesaplanmıştır. Son aşamada bu altı kalite niteliği toplanarak Total Quality Index (TQI) değeri elde edilmiştir.

3. Yöntem

3.1. Seçilen Yazılım Sistemi

Analiz için Java tabanlı açık kaynak bir komut satırı arayüzü kütüphanesi olan picocli seçilmiştir. Projenin çok sayıda sürüme sahip olması, Java ile geliştirilmiş olması ve statik analiz araçlarıyla incelenebilir olması nedeniyle QMOOD tabanlı sürüm analizi için uygun olduğu değerlendirilmiştir.

3.2. İncelenen Sürümler

Tablo 1. İncelenen picocli sürümleri

Sıra	Sürüm
1	v4.0.4
2	v4.1.0
3	v4.1.4
4	v4.2.0

Sıra	Sürüm
5	v4.3.0
6	v4.3.2
7	v4.4.0
8	v4.5.2
9	v4.6.3
10	v4.7.7

3.3. Metrik Çıkarım Süreci

Her sürümün kaynak kodu ayrı klasöre çıkarılmış ve yalnızca src/main/java dizini analiz edilmiştir. Test kodları, örnek projeler, dokümantasyon ve build çıktıları analize dahil edilmemiştir. Metrik çıkarımı CK Tool ile yapılmış, her sürüm için class.csv ve method.csv dosyaları oluşturulmuştur.

Ham metriklerden sürüm bazlı özet tablolar üretilmiştir. Bu özetlerde sınıf sayısı, metot sayısı, ortalama CBO, ortalama WMC, ortalama DIT, ortalama RFC, ortalama LCOM ve DAM değeri kullanılmıştır.

3.4. QMOOD Tasarım Özelliklerine Dönüşüm

CK Tool çıktıları QMOOD'un özgün metriklerinin tamamını doğrudan sağlamadığı için bazı tasarım özellikleri için vekil metrikler kullanılmıştır. Bu durum çalışmanın sınırlılığı olarak değerlendirilmiş ve aşağıdaki tabloda açıkça belirtilmiştir.

Tablo 2. QMOOD tasarım özellikleri için kullanılan metrik/proxy eşleştirmesi

QMOOD Özelliği	Kullanılan Metrik	Açıklama
Design Size	class_count	Toplam sınıf sayısı
Hierarchies	classes_with_children	Çocuk sınıf olan sınıf sayısı
Abstraction	avg_dit	Ortalama DIT
Encapsulation	dam	Private/protected alan oranı
Coupling	avg_cbo	Ortalama CBO
Cohesion	$1 / (1 + \text{avg_lcom})$	LCOM üzerinden ters cohesion göstergesi
Composition	total_fields_qty	Alan sayısı üzerinden yaklaşık gösterge
Inheritance	avg_dit	Kalıtım derinliği üzerinden yaklaşık gösterge
Polymorphism	classes_with_children	Kalıtım yapısı üzerinden yaklaşık gösterge
Messaging	public_methods_qty	Public metot sayısı
Complexity	avg_wmc	Ortalama WMC

Normalizasyon işleminde v4.0.4 sürümü referans alınmıştır. Böylece elde edilen değerler mutlak kalite düzeyini değil, başlangıç sürümüne göre göreceli kalite değişimini ifade eder.

4. Analiz Süreci

Analiz süreci dört aşamada yürütülmüştür. İlk olarak picocli deposundaki seçilen tag sürümleri ayrı klasörlere çıkarılmıştır. İkinci aşamada CK Tool ile her sürümün src/main/java dizini analiz edilmiştir. Üçüncü aşamada Python ile ham class-level ve method-level CSV çıktıları sürüm bazlı özet metriklere dönüştürülmüştür. Son aşamada QMOOD formülleri uygulanarak kalite nitelikleri ve TQI değerleri hesaplanmış, Matplotlib ile grafikler üretilmiştir.

5. Sonuçlar

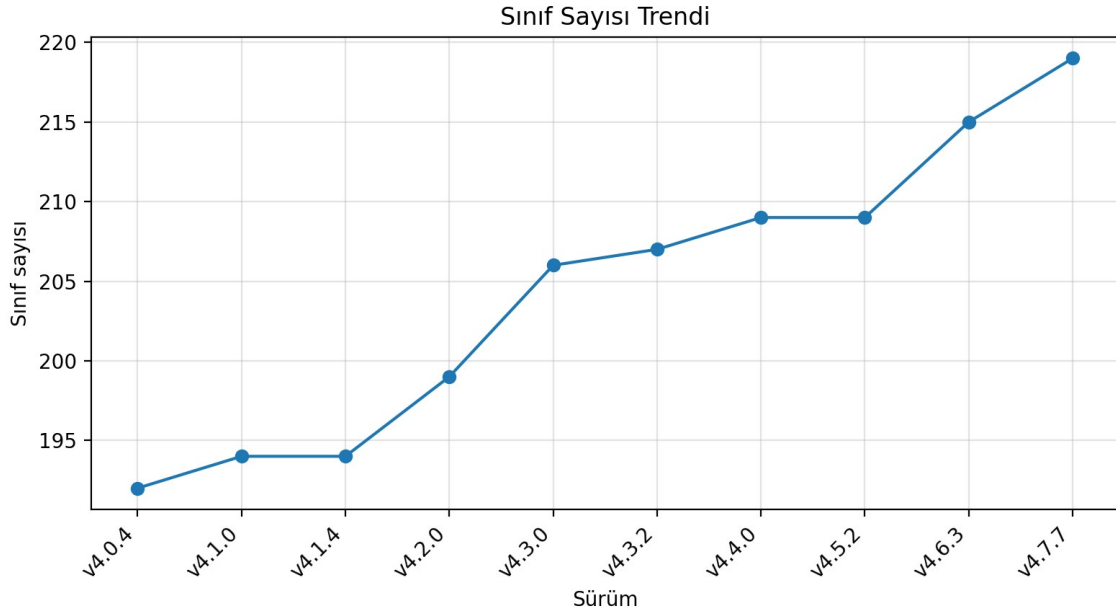
5.1. Temel Metrik Sonuçları

Tablo 3. Sürüm bazlı özet metrik sonuçları

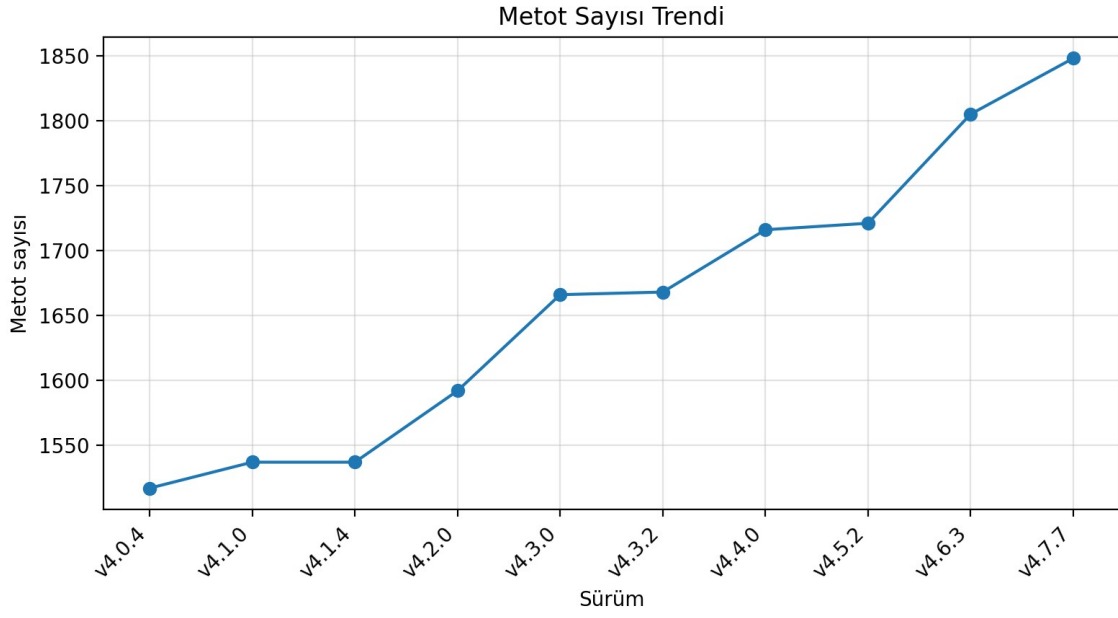
Sürüm	Sınıf	Metot	Avg CBO	Avg WMC	Avg DIT	Avg RFC	Avg LCOM	DAM
v4.0.4	192	1517	4.9219	17.4048	1.8125	14.2969	157.80	0.7739

Sürüm	Sınıf	Metot	Avg CBO	Avg WMC	Avg DIT	Avg RFC	Avg LCOM	DAM
v4.1.0	194	1537	4.9485	17.6031	1.8041	14.4485	160.19	0.7737
v4.1.4	194	1537	4.9558	17.6959	1.8041	14.5052	160.93	0.7737
v4.2.0	199	1592	4.9950	18.0000	1.7839	14.7437	165.29	0.7723
v4.3.0	206	1666	4.9854	18.5097	1.7700	15.0971	173.64	0.7743
v4.3.2	207	1668	4.9807	18.4203	1.7826	15.0338	173.04	0.7727
v4.4.0	209	1716	4.9569	18.7895	1.7943	15.2105	185.55	0.7758
v4.5.2	209	1721	4.9809	18.8325	1.7943	15.2919	185.23	0.7758
v4.6.3	215	1805	5.0093	19.2419	1.7721	15.5163	209.29	0.7737
v4.7.7	219	1848	5.0411	19.5616	1.7580	15.7763	214.35	0.7790

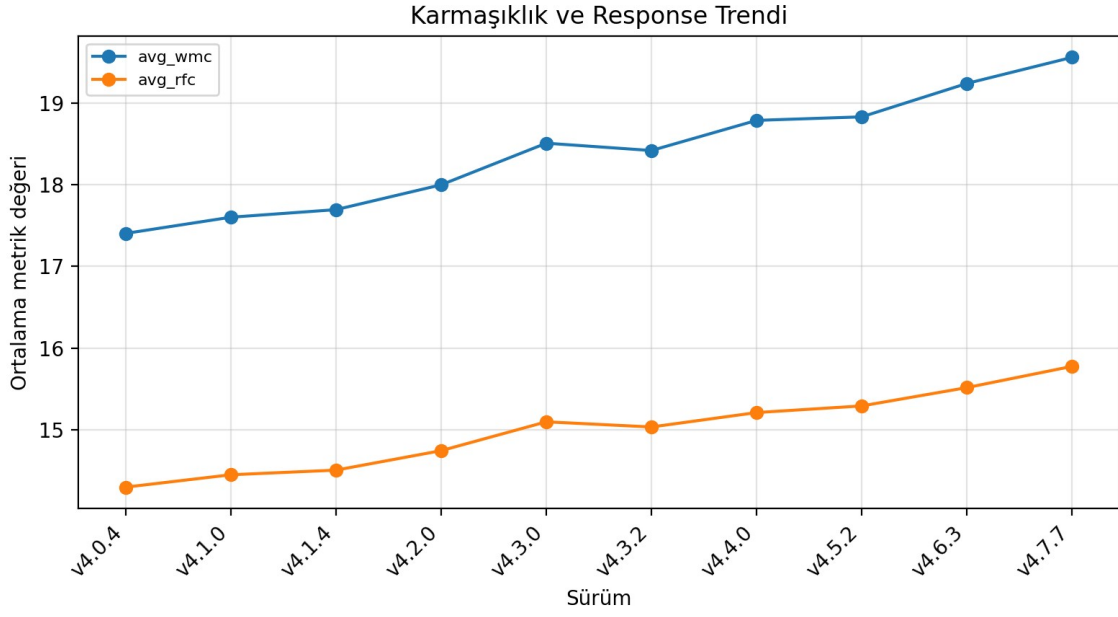
Tablo 3 incelendiğinde sınıf sayısının 192'den 219'a, metot sayısının ise 1517'den 1848'e yükseldiği görülmektedir. Bu artış, picocli projesinin incelenen sürümler boyunca işlevsel olarak büyüdüğünü göstermektedir. Ortalama WMC ve RFC değerlerinin artması, sınıf düzeyinde karmaşıklığın ve yanıt kümesinin genişlediğini düşündürmektedir. Ortalama LCOM değerindeki artış ise cohesion açısından dikkat edilmesi gereken bir eğilimdir.



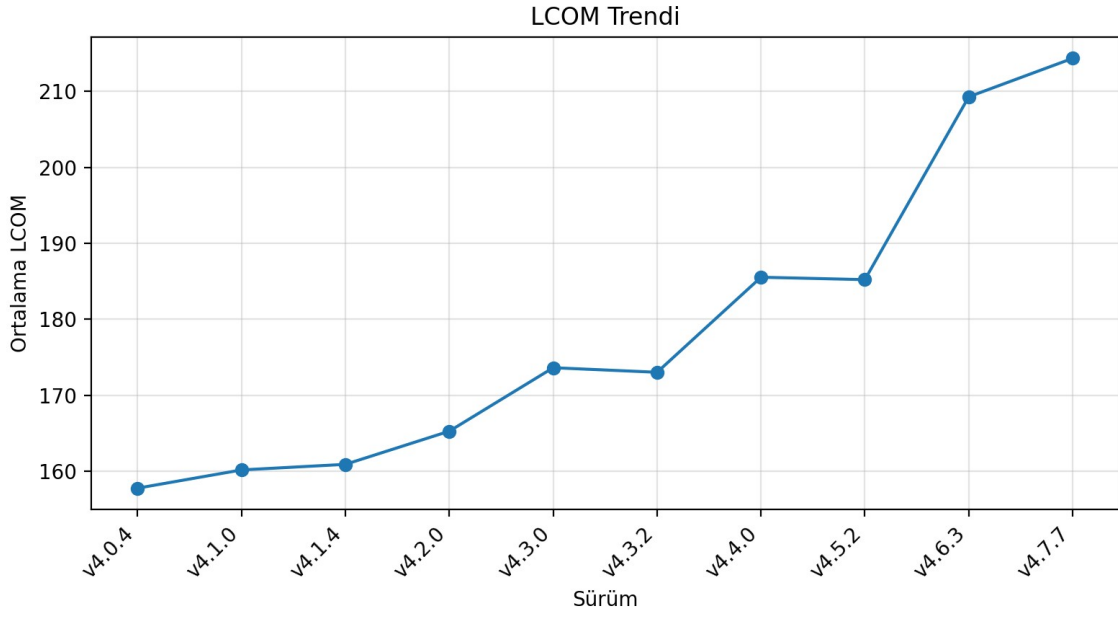
Şekil 1. Picocli sınıf sayısı trendi



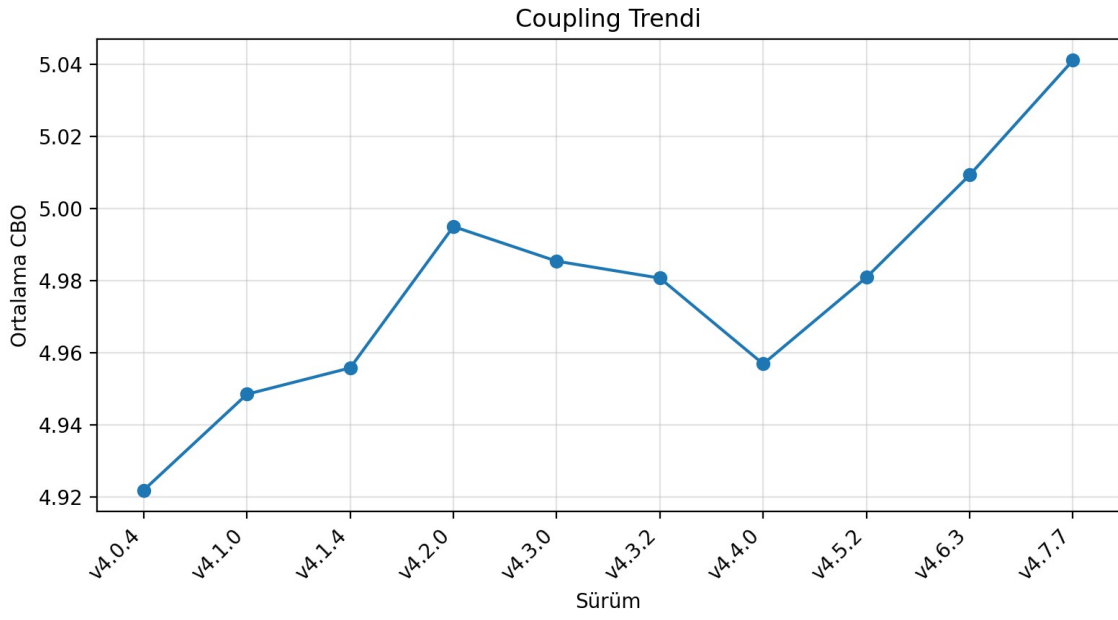
Şekil 2. Picocli metot sayısı trendi



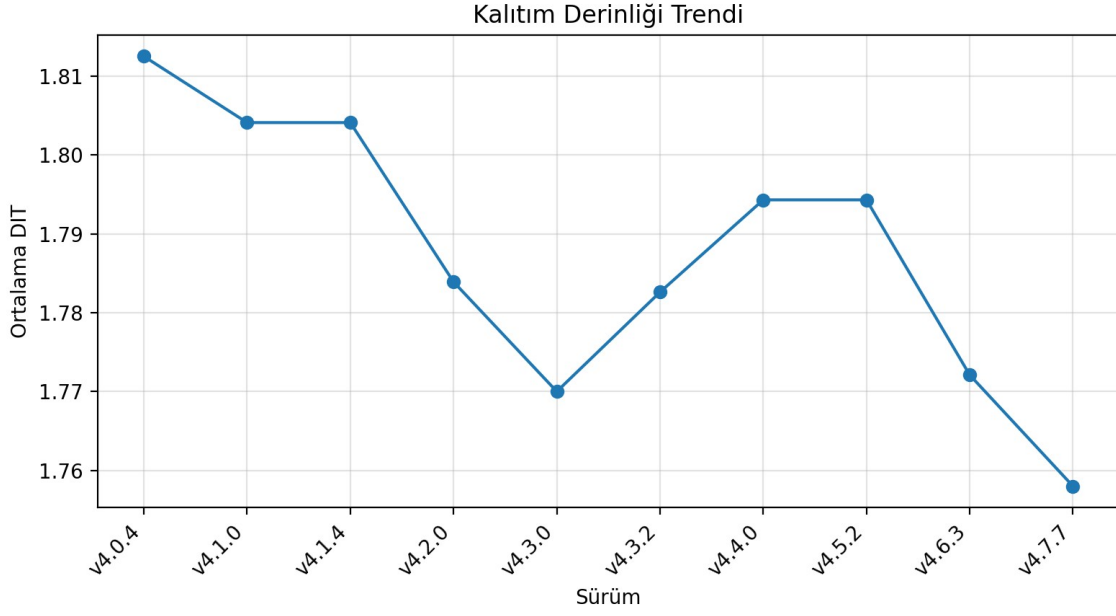
Şekil 3. Ortalama WMC ve RFC trendi



Şekil 4. Ortalama LCOM trendi



Şekil 5. Ortalama CBO (coupling) trendi



Şekil 6. Ortalama DIT (kalıtım derinliği) trendi

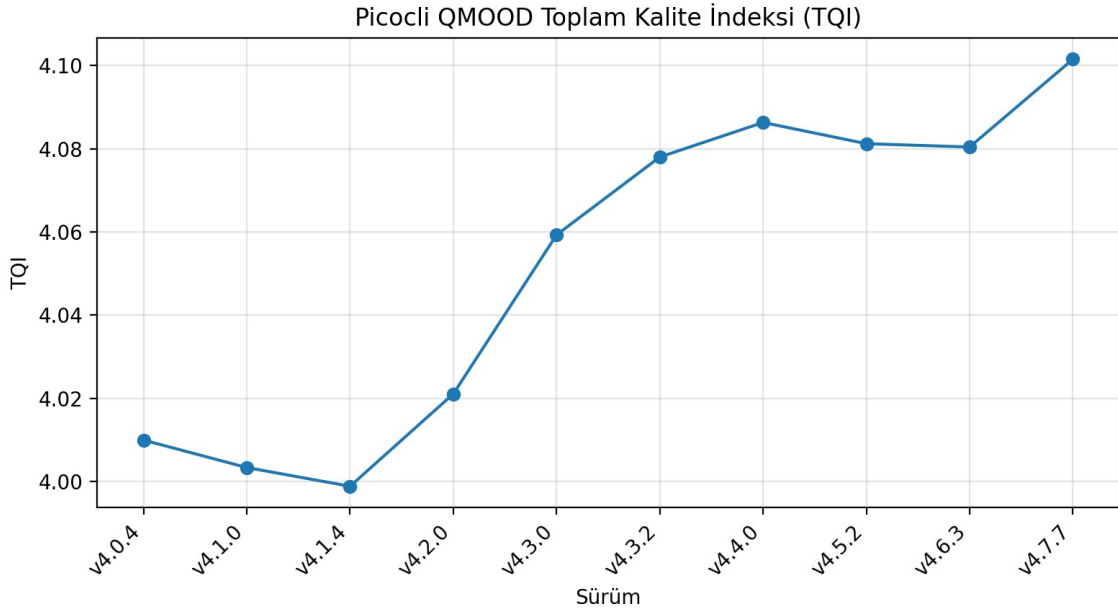
Coupling trendi sınıflar arası bağımlılığın sürümler boyunca tamamen kontrolsüz biçimde artmadığını, ancak son sürümlerde hafif bir yükseliş olduğunu göstermektedir. DIT trendi ise kalıtım derinliğinin büyük ölçüde sabit kaldığını ve hatta son sürümlerde hafifçe düştüğünü ortaya koymaktadır. Bu durum, büyüyen sisteme rağmen kalıtım yapısının aşırı karmaşıklaşmadığını düşündürür.

5.2. QMOOD Kalite Skorları

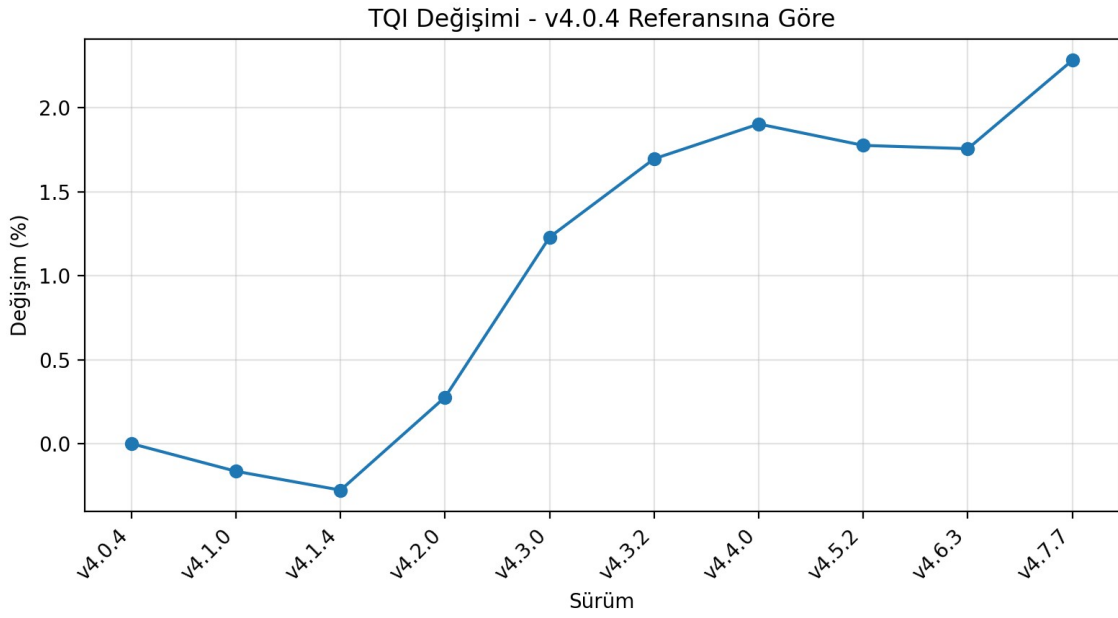
Tablo 4. QMOOD kalite skorları

Sürüm	Reusability	Flexibility	Understandability	Functionality	Extendibility	Effectiveness	TQI
v4.0.4	1.0000	1.0000	-0.9900	1.0000	1.0000	1.0000	4.0100
v4.1.0	1.0063	1.0029	-1.0015	1.0032	0.9927	0.9998	4.0034
v4.1.4	1.0058	1.0024	-1.0040	1.0032	0.9916	0.9998	3.9989
v4.2.0	1.0319	1.0208	-1.0268	1.0152	0.9768	1.0033	4.0211
v4.3.0	1.0655	1.0458	-1.0593	1.0293	0.9684	1.0096	4.0593
v4.3.2	1.0705	1.0466	-1.0611	1.0314	0.9775	1.0131	4.0780
v4.4.0	1.0803	1.0564	-1.0910	1.0346	0.9864	1.0195	4.0863
v4.5.2	1.0809	1.0552	-1.0938	1.0354	0.9840	1.0195	4.0812
v4.6.3	1.0927	1.0911	-1.1419	1.0403	0.9688	1.0293	4.0804
v4.7.7	1.1132	1.1054	-1.1579	1.0498	0.9578	1.0332	4.1015

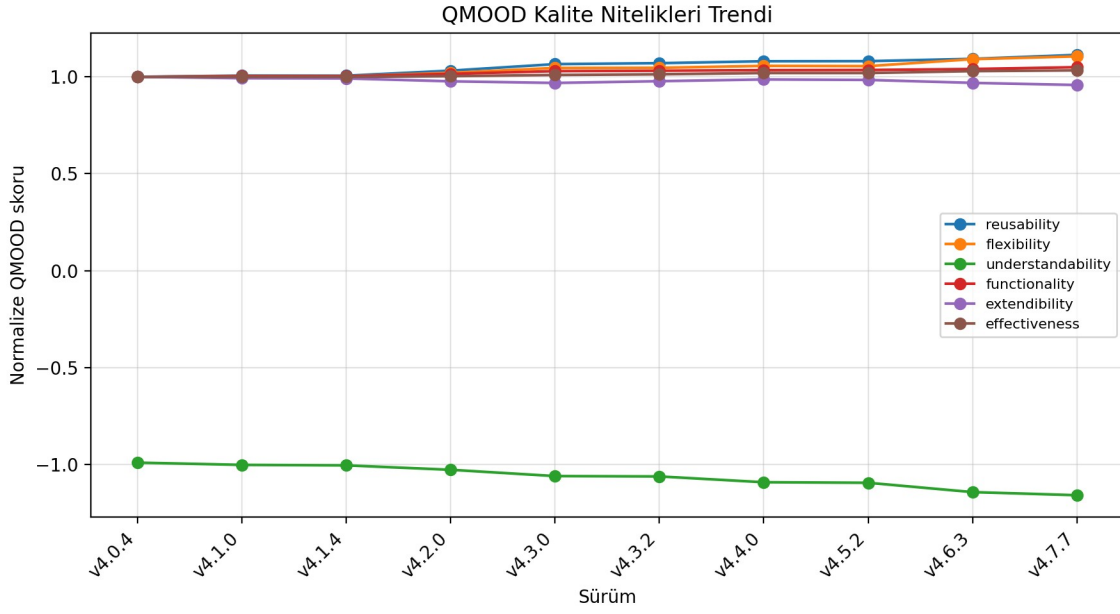
QMOOD skorları incelendiğinde TQI değerinin v4.0.4 sürümünde 4.0100, v4.7.7 sürümünde ise 4.1015 olduğu görülmektedir. Bu değişim yaklaşık %2.28'lik görece artışa karşılık gelir. Artışın temel kaynağı reusability, flexibility, functionality ve effectiveness niteliklerindeki yükselmedir. Buna karşılık understandability değeri -0.9900'dan -1.1579'a düşmüştür.



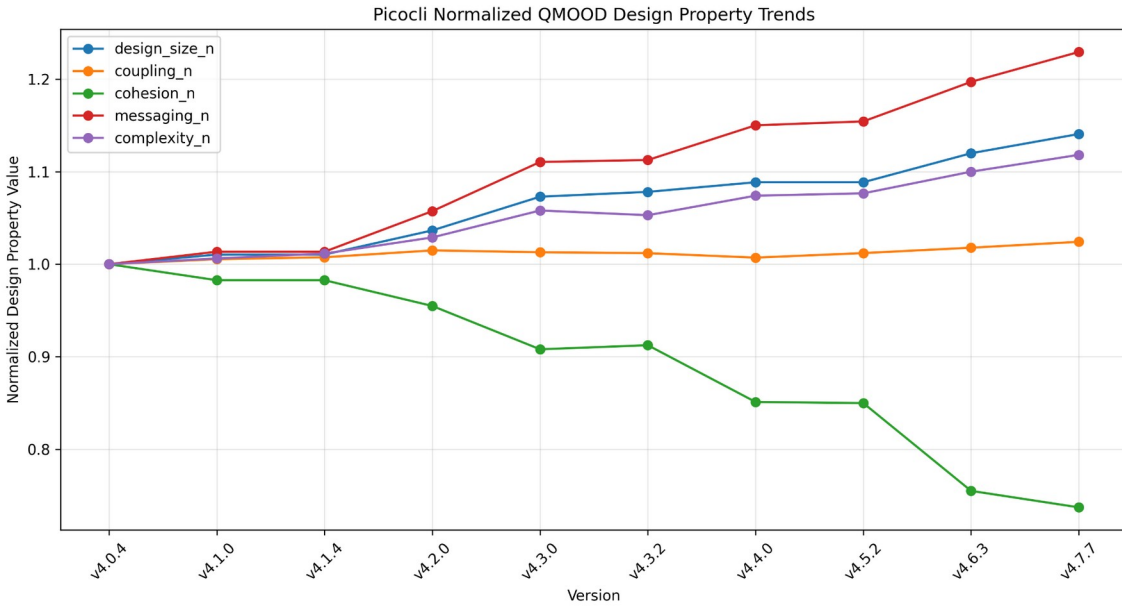
Şekil 7. QMOOD Total Quality Index (TQI) trendi



Şekil 8. v4.0.4 referansına göre TQI yüzde değişimi



Şekil 9. QMOOD kalite nitelikleri trendi



Şekil 10. Normalize QMOOD tasarım özellikleri trendi

Normalize QMOOD tasarım özellikleri trendi incelendiğinde design size, messaging ve complexity göstergelerinin arttığı; cohesion göstergesinin ise LCOM artışına paralel olarak düştüğü görülmektedir. Coupling göstergesi daha sınırlı bir artış sergilemektedir. Bu sonuç, sistemin işlevsel ve yapısal olarak genişlerken anlaşılabilirlik ve iç uyum açısından baskı altında kaldığını desteklemektedir.

5.3. Bulguların Özeti

- Picocli incelenen sürümler boyunca büyümüştür: sınıf sayısı yaklaşık %14.1, metot sayısı yaklaşık %21.8 artmıştır.
- TQI değeri 4.0100'dan 4.1015'e yükselmiştir; bu artış yaklaşık %2.28 düzeyindedir.
- Reusability ve flexibility değerleri artmıştır. Bu durum public servislerin ve tasarımın kullanım/uyarlanabilirlik kapasitesinin yükseldiğini düşündürmektedir.

- Understandability değeri düşmüştür. Bu düşüş, WMC, RFC ve LCOM artışlarıyla birlikte yorumlandığında karmaşıklık ve bakım riski oluşturabilir.
- DIT değerinde belirgin artış görülmemiştir; bu durum kalıtım derinliğinin kontrol altında kaldığını göstermektedir.
- Extendibility değerinde hafif düşüş gözlenmiştir. Bu durum coupling ve kullanılan proxy metriklerin etkisiyle yorumlanmalıdır.

6. Tartışma

6.1. Yazılım Evrimi ve Kalite Dengesi

Analiz sonuçları, picocli projesinin işlevsel olarak genişlediğini ancak bu genişlemenin anlaşılabilirlik üzerinde olumsuz bir etki yarattığını göstermektedir. Yazılım evriminde sık karşılaşılan bu durum, yeni özelliklerin ve servislerin eklenmesiyle birlikte sınıf sorumluluklarının genişlemesi ve metotlar arası uyumun azalması şeklinde yorumlanabilir.

6.2. Mimari Bozulma ve Teknik Borç Göstergeleri

Mimari bozulma açısından en önemli gösterge LCOM değerindeki artıştır. LCOM artışı, bazı sınıfların birden fazla sorumluluk taşımaya başlamış olabileceğini gösterir. Bu nedenle özellikle yüksek LCOM, yüksek WMC ve yüksek RFC değerlerine sahip sınıflar ayrıntılı olarak incelenmelidir. Bu sınıflar için Extract Class, Extract Method, sorumluluk ayırıştırma ve arayüz sadeleştirme gibi refactoring teknikleri önerilebilir.

Coupling değerinde çok keskin bir artış gözlenmemiştir; ancak son sürümlerde ortalama CBO değerinin 5.0411'e yükselmesi modülerlik açısından izlenmelidir. Bağımlılığı yüksek sınıflarda facade/controller sorumluluklarının gereğinden fazla büyüüp büyümediği kontrol edilmelidir.

6.3. Sınırlılıklar

Bu çalışmada QMOOD'un özgün metriklerinin tamamı doğrudan elde edilemediği için bazı tasarım özellikleri proxy metriklerle temsil edilmiştir. Cohesion için TCC yerine LCOM tabanlı ters dönüşüm kullanılmıştır. Composition, inheritance ve polymorphism için de CK Tool çıktılarından türetilmiş yaklaşık göstergeler kullanılmıştır. Bu nedenle elde edilen QMOOD skorları mutlak kalite hükmü olarak değil, seçilen yöntem kapsamında sürümler arası göreceli değişim göstergesi olarak değerlendirilmelidir.

6.4. LLM Destekli Değerlendirme İçin Hazırlık

Proje yönergesi en az üç farklı LLM modelinden kalite değerlendirmesi alınmasını istemektedir. Bu çalışmada QMOOD ve CK metrik sonuçları aynı ana prompt ile Gemini, DeepSeek ve Claude modellerine verilmiştir. Modellerden genel kalite değerlendirmesi, bakım yapılabilirlik analizi, teknik borç yorumu, mimari kalite yorumu ve refactoring önerileri istenmiştir. Elde edilen yanıtlar metriklerle bağlılık, teknik doğruluk, tutarlılık, refactoring kalitesi, halüsinasyon riski, trend analizi ve sınırlılık farkındalığı ölçütleriyle eleştirel olarak değerlendirilmiştir. Kullanılan promptlar ve değerlendirme rubriği Ek A'da verilmiştir.

6.5. LLM Çıktılarının Karşılaştırmalı Analizi

LLM deneyinde üç farklı modelden alınan cevaplar karşılaştırılmıştır: Gemini, DeepSeek ve Claude. Aynı veri seti ve aynı ana prompt kullanıldığı için modellerin cevapları doğrudan karşılaştırılabilir hale getirilmiştir. Cevapların tamamı TQI değerindeki sınırlı artışı, Reusability/Flexibility/Functionality artışını ve Understandability düşüşünü yakalamıştır. Bununla birlikte modellerin bazıları verilen ortalama metriklerin ötesine geçerek God Class, SRP/OCF ihlali veya composition kullanımının artması gibi doğrudan kanıtlanamayan çıkarımlar yapmıştır. Bu nedenle LLM çıktıları nihai kanıt olarak değil, metrik tabanlı analizleri yorumlamaya yardımcı nitel değerlendirme çıktıları olarak kullanılmıştır.

Model	Metriklere bağlılık	Teknik doğruluk	Refactoring kalitesi	Halüsinasyon kontrolü	Trend analizi	Genel puan
Gemini	5	4	4	3	5	21/25

DeepSeek	5	3	3	3	5	19/25
Claude	5	4	4	3	5	21/25

Tablo 5. LLM model çıktılarının rubrik temelli karşılaştırması

Tablo 5'e göre Gemini ve Claude genel puanda en yüksek değeri almıştır. Gemini, metriklerle uyumlu ve dengeli yorumlar üretirken; Claude, akademik anlatım ve sınırlılık farkındalığı açısından güçlü bir çıktı vermiştir. DeepSeek ise proxy metrikler, normalizasyon ve ortalama değerlerin sınıf düzeyindeki uç değerleri gizleyebileceği gibi sınırlılıkları en ayrıntılı ele alan model olmuştur. Buna rağmen DeepSeek'in DAM metriğini soyutlama göstergesi gibi yorumlaması teknik doğruluk puanını düşürmüştür. DAM bu çalışmada encapsulation için kullanılan veri erişim/kapsülleme oranıdır.

Model	Güçlü yönler	Eleştirel notlar
Gemini	TQI artışını sınırlı yorumladı; WMC, RFC ve LCOM artışını bakım riskiyle ilişkilendirdi; CBO artışını abartmadı.	God Class ve Lehman kanunları gibi verilen veriden doğrudan kanıtlanamayan kavramlara yer verdi.
DeepSeek	TQI'nin tek başına yanıltıcı olabileceğini, normalizasyon ve proxy metrik sınırlılıklarını güçlü biçimde vurguladı.	DAM metriğini hatalı biçimde soyutlama göstergesi gibi yorumladı; OCP/SRP çıkarımlarında fazla kesin konuştu.
Claude	En akademik ve yapılandırılmış yanıtı verdi; DAM/encapsulation yorumunu doğru yaptı; sınırlılıkları iyi belirtti.	God Class, big ball of mud, OCP ihlali ve composition'a yönelme gibi çıkarımları ortalama metriklerle kesin kanıtlanamaz.

Tablo 6. LLM cevaplarının nitel değerlendirme özeti

6.5.1. Model Bazlı Yorumlar

Gemini cevabı, metriklerle bağılılık açısından güçlüdür. Class count, method count, WMC, RFC, LCOM, CBO, DIT ve TQI değerlerini açıkça kullanmıştır. Model, TQI artışının genel kalite artışı olarak tek başına yorumlanmaması gerektiğini belirtmiş ve Understandability düşüşünü WMC/RFC/LCOM artışıyla ilişkilendirmiştir. Ancak God Class ve Lehman kanunları gibi bazı ifadeleri verilen veri setinden doğrudan kanıtlanamadığı için raporda yumuşatılarak kullanılmalıdır.

DeepSeek cevabı, sınırlılık farkındalığı açısından güçlüdür. Proxy metrikler, v4.0.4 referansına göre normalizasyon, ortalama değerlerin aykırı sınıfları gizleyebilmesi ve test kapsamı eksikliği gibi noktaları açıkça tartışmıştır. Bununla birlikte DAM metriğini "sınıfların ne kadar soyut olduğu" şeklinde yorumlaması teknik bir hatadır. DAM, QMOOD bağlamında veri erişimi/kapsülleme ile ilişkilidir. Bu nedenle DeepSeek çıktısı raporda düzeltme notuyla kullanılmalıdır.

Claude cevabı, anlatım yapısı ve akademik üslup bakımından güçlüdür. DAM değerinin 0.773-0.779 aralığında kararlı kalmasını veri gizleme/encapsulation pratiğiyle ilişkilendirmesi doğrudur. Model ayrıca TQI artışının Understandability ve Extendibility düşüşünü perdeleyebileceğini açık biçimde belirtmiştir. Ancak SRP/OCP ihlali, God Class riski ve composition'a yönelme gibi yorumlar sınıf düzeyi veri veya kod diff analizi olmadan kesin sonuç olarak sunulmamalıdır.

6.5.2. Ortak Bulgular ve Ayrışmalar

Üç modelin ortak bulgusu, picocli'nin incelenen sürümler boyunca işlevsel olarak büyüdüğü ve TQI değerinin küçük fakat pozitif bir artış gösterdiğidir. Modellerin tamamı Reusability, Flexibility, Functionality ve Effectiveness niteliklerindeki artış olumlu; Understandability ve Extendibility değerlerindeki düşüşü ise bakım yapılabilirlik açısından riskli yorumlamıştır. Ayrışma noktası, modellerin metriklerden ne kadar ileri çıkarım yaptığıdır. Gemini ve Claude bazı kavramsal genellemelere giderken, DeepSeek sınırlılıkları daha fazla vurgulamış ancak DAM yorumunda hata yapmıştır.

LLM sonuçları QMOOD analizini genel olarak desteklemektedir: LCOM artışı cohesion kaybı, WMC/RFC artışı karmaşıklık ve test edilebilirlik riski, CBO'daki sınırlı artış ise coupling'in henüz kontrol dışına çıkmadığı şeklinde yorumlanmıştır. Bununla birlikte LLM'lerin ürettiği sonuçlar sınıf düzeyi kanıtlarla desteklenmedikçe nihai mimari teşhis olarak kabul edilmemelidir.

6.5.3. Refactoring Önerilerinin Birleştirilmiş Özeti

Öncelik	Birleştirilmiş refactoring önerisi	Dayandığı metrik	Beklenen kalite etkisi
1	Yüksek LCOM değerine sahip sınıfları belirleyip Extract Class uygulamak	LCOM artışı: 157.7969 -> 214.3470	Cohesion ve Understandability
2	Yüksek WMC/RFC değerlerine sahip sınıflarda Extract Method ve metod sadeleştirme yapmak	WMC: 17.4048 -> 19.5616; RFC: 14.2969 -> 15.7763	Understandability, test edilebilirlik
3	Gereksiz public metodları azaltmak ve sınıf arayüzlerini sadeleştirmek	RFC ve public arayüz büyümesi	Messaging kontrolü, bakım kolaylığı
4	Yüksek CBO değerine sahip sınıfları ayrıca incelemek	CBO sınırlı artsa da son sürümde 5.0411	Coupling ve modülerlik
5	Yeni özellik ekleme öncesinde refactoring sprint'i planlamak	TQI artarken Understandability ve Extendibility düşüyor	Sürdürülebilir kalite evrimi

Tablo 7. LLM çıktılarından türetilen ortak refactoring öncelikleri

Bu öneriler LLM çıktılarının ortaklaştığı noktalardan oluşturulmuştur. Bu aşamanın ardından yapılan v4.7.7 sınıf düzeyi risk analizi, önerilerin özellikle picocli.CommandLine, CommandSpec, Interpreter ve Help çevresinde somutlaştırılabileceğini göstermiştir. Böylece LLM tabanlı genel refactoring önerileri sınıf düzeyi metriklerle desteklenmiştir.

6.5.4. LLM Değerlendirmesinin Sonucu

LLM destekli değerlendirme, QMOOD sonuçlarının yorumlanmasında yararlı bir ikinci bakış sağlamıştır. Modeller, metrik trendlerini büyük ölçüde doğru okumuş ve teknik borç riskini özellikle cohesion ve karmaşıklık üzerinden açıklamıştır. Ancak bazı modellerin metriklerin ötesinde kesin mimari teşhisler üretme eğilimi, LLM çıktılarının eleştirel süzgeçten geçirilmesi gerektiğini göstermiştir. Bu nedenle çalışmada LLM sonuçları, ölçülmüş metriklerin yerine geçen bir kanıt olarak değil, metrik tabanlı bulguların yorumlanmasını destekleyen karşılaştırmalı nitel analiz çıktıları olarak kullanılmıştır.

6.6. Prompt 2 ile Yapılandırılmış LLM Değerlendirmesi

Prompt 2, modellerin daha kısa ve aynı başlıklar altında karşılaştırılabilir cevaplar üretmesini sağlamak için kullanılmıştır. Bu promptta modellere yalnızca v4.0.4 ve v4.7.7 arasındaki özet metrik değişimleri verilmiş; genel kalite eğilimi, iyileşen/kötüleşen kalite nitelikleri, bakım yapılabilirlik riski, teknik borç göstergeleri, mimari bozulma işaretleri, refactoring önerileri ve modelin emin olmadığı noktaları aynı formatta açıklamaları istenmiştir.

Tablo 8. Prompt 2 çıktılarının rubrik temelli karşılaştırması

Model / Prompt	Metriklere bağlılık	Teknik doğruluk	Refactoring kalitesi	Halüsinasyon riski	Trend analizi	Genel puan
Gemini - Prompt 2	5	4	4	3	5	21/25
DeepSeek - Prompt 2	5	3	2	2	5	17/25
Claude - Prompt 2	5	4	4	3	5	22/25

Prompt 2 sonuçlarına göre Claude en dengeli yapılandırılmış cevabı üretmiştir. Claude, DAM metriğini doğru biçimde veri gizleme/encapsulation ile ilişkilendirmiş, QMOOD skorlarının referans sürüme göre normalize edildiğini belirtmiş ve sınıf düzeyi veri eksikliğini açıkça vurgulamıştır. Gemini de metriklerle bağlı ve tutarlı bir analiz sunmuştur; ancak DIT düşüşünden polimorfizm azalması veya mevcut sınıflara kod yığılması gibi doğrudan kanıtlanamayan çıkarımlar yapmıştır. DeepSeek ise TQI, LCOM ve Understandability trendlerini doğru yakalamış olsa da DAM metriğini abstraction göstergesi olarak yorumladığı ve sınıf düzeyi veri olmadan Feature Envy gibi sonuçlara yöneldiği için teknik doğruluk açısından daha zayıf kalmıştır.

Tablo 9. Prompt 2 cevaplarının nitel özeti

Model	Güçlü yön	Dikkat edilmesi gereken nokta
Gemini	TQI artışının dengesiz olduğunu, LCOM/WMC/RFC artışının bakım riskini yükselttiğini doğru yorumladı.	DIT ve polimorfizm ilişkisi ile "sınıflara kod yığılması" yorumu doğrudan kanıtlanamaz.
DeepSeek	TQI artışının Understandability/Extendibility düşüşünü maskeleyebileceğini ve normalizasyon sınırlılığını iyi belirtti.	DAM metriğini abstraction göstergesi gibi ele aldı; Extract Interface-DAM ilişkisi teknik olarak sorunlu.
Claude	En dengeli çıktı; DAM yorumu doğru, sınırlılıklar açık, refactoring önerileri uygulanabilir.	SRP/composition çıkarımları sınıf düzeyi veri olmadan kesin kabul edilmemeli.

Prompt 2 ile alınan yapılandırılmış cevaplar, Prompt 1 sonuçlarını genel olarak desteklemiştir. Üç model de TQI artışını tek başına yeterli görmemiş; Reusability, Flexibility, Functionality ve Effectiveness artarken Understandability ve Extendibility değerlerinin gerilediğini vurgulamıştır. Ayrıca modellerin tamamı LCOM artışını temel teknik borç göstergesi olarak değerlendirmiştir. Buna karşılık, LLM çıktılarında bazı metriklerden doğrudan kanıtlanamayan çıkarımlar da görülmüştür. Bu

nedenle LLM sonuçları QMOOD analizini yorumlamada yararlı bir destek olarak kullanılmış, ancak nihai kararlar metriklerin kendisi ve yöntemsel sınırlılıklar dikkate alınarak verilmiştir.

6.7. v4.7.7 Sınıf Düzeyi Risk Analizi

Sürüm ortalamaları genel eğilimi göstermesine rağmen riskin hangi sınıflarda yoğunlaştığını ortaya koymaz. Bu nedenle v4.7.7 sürümü için CK Tool class.csv çıktısı sınıf düzeyinde incelenmiş ve normalize edilmiş birleşik risk skoru hesaplanmıştır. Risk skoru LCOM, WMC, RFC, CBO ve DIT metriklerinden türetilmiştir; ağırlıklar sırasıyla LCOM için 0.35, WMC için 0.25, RFC için 0.20, CBO için 0.15 ve DIT için 0.05 olarak belirlenmiştir. Bu ağırlıklandırmada cohesion kaybı ve sınıf içi karmaşıklık, bakım yapılabilirlik açısından daha yüksek öneme sahip kabul edilmiştir.

Tablo 10. v4.7.7 sürümü için risk skoru en yüksek ilk 10 sınıf

Sıra	Sınıf	WMC	RFC	LCOM	CBO	DIT	Risk Skoru
1	picocli.CommandLine	362	338	17878	66	76	0.893
2	picocli.CommandLine\$Model\$CommandSpec	388	324	10708	50	1	0.674
3	picocli.CommandLine\$Interpreter	527	375	2032	70	1	0.640
4	picocli.CommandLine\$Help	207	245	2635	40	2	0.367
5	picocli.CommandLine\$Model\$ArgSpec	184	109	2090	31	1	0.253
6	picocli.CommandLine\$Model\$UsageMessageSpec	206	111	2809	14	1	0.242
7	picocli.CommandLine\$Model\$ArgGroupSpec	137	114	741	34	1	0.213
8	picocli.AutoComplete	156	132	778	24	1	0.211
9	picocli.CommandLine\$Model\$CommandReflection	101	135	153	35	1	0.198
10	picocli.CommandLine\$Model\$ArgSpec\$Builder	97	81	2215	27	1	0.190

Tablo 10 incelendiğinde riskin proje geneline eşit dağılmadığı, özellikle picocli.CommandLine ve onun iç sınıfları çevresinde yoğunlaştığı görülmektedir. En yüksek risk skoruna sahip sınıf picocli.CommandLine olmuştur. Bu sınıfın WMC, RFC, LCOM ve CBO değerlerinin oldukça yüksek olması, sınıfın hem karmaşıklık hem cohesion hem de coupling açısından kritik bir refactoring adayı olduğunu göstermektedir. CommandLine\$Model\$CommandSpec ve CommandLine\$Interpreter sınıfları da yüksek WMC ve RFC değerleri nedeniyle bakım yapılabilirlik açısından öncelikli incelenmesi gereken sınıflar arasındadır. Bu bulgu, sürüm ortalamalarında görülen LCOM, WMC ve RFC artışlarının özellikle belirli merkezi sınıflarda yoğunlaşmış olabileceğini göstermektedir.

Refactoring açısından ilk öncelik, picocli.CommandLine sınıfının sorumluluklarının ayrıştırılmasıdır. Yüksek LCOM değerleri Extract Class, yüksek WMC ve RFC değerleri ise Extract Method, arayüz sadeleştirme ve ayrıştırılmış yardımcı bileşenler kullanma ihtiyacına işaret etmektedir. Bununla birlikte picocli.CommandLine için DIT değerinin 76 olarak görünmesi, CK Tool'un iç/nested class yapısını yorumlama biçiminden etkilenmiş olabilecek aykırı bir değer olarak ayrıca doğrulanmalıdır. Dolayısıyla bu değer tek başına kalıtım hiyerarşisinin gerçek derinliği olarak değil, sınıf düzeyi risk taramasında dikkat çekici bir sinyal olarak değerlendirilmiştir.

7. Sonuç ve Gelecek Çalışmalar

Bu çalışmada picocli projesinin 10 sürümü üzerinde CK Tool ile metrik çıkarımı yapılmış, bu metrikler QMOOD kalite modeline göre değerlendirilmiş ve sonuçlar Gemini, DeepSeek ve Claude olmak üzere üç farklı LLM modeliyle karşılaştırmalı olarak yorumlanmıştır. Genel sonuç, yazılımın v4.0.4 ile v4.7.7 arasında büyürken TQI değerinde küçük fakat pozitif bir artış gösterdiği. Bu artış reusability, flexibility, functionality ve effectiveness niteliklerindeki iyileşmeyle ilişkilidir. LLM çıktıları da

bu genel eğilimi doğrulamış, ancak TQI'nin tek başına kullanılmasının bazı kalite düşüşlerini perdeleyebileceğine dikkat çekmiştir.

Buna karşılık understandability ve extendibility değerlerindeki düşüş, yazılımın büyümesine bağlı olarak karmaşıklık ve cohesion risklerinin arttığını göstermektedir. Özellikle WMC, RFC ve LCOM metriklerinde yükseliş gözlenmiştir. LLM değerlendirmeleri de aynı riskleri öne çıkarmış; ancak God Class, SRP/OCP ihlali veya composition kullanımına ilişkin bazı çıkarımların sınıf düzeyi veri olmadan kesin kabul edilemeyeceği görülmüştür. Gelecek çalışmalarda yüksek riskli sınıfların sınıf bazında incelenmesi, refactoring etkilerinin yeni metrik ölçümleriyle doğrulanması ve LLM çıktılarının daha sıkı bir rubrikle tekrar değerlendirilmesi önerilmektedir.

v4.7.7 için yapılan sınıf düzeyi risk analizi, teknik borç göstergelerinin özellikle picocli.CommandLine ve onun iç sınıfları çevresinde yoğunlaştığını ortaya koymuştur. Bu nedenle gelecekteki iyileştirme çalışmalarında öncelik, CommandLine ana sınıfı ile Model, Interpreter, Help ve ParseResult alt yapılarındaki yüksek WMC, RFC ve LCOM değerlerinin düşürülmesine verilmelidir.

Prompt 2 ile yapılan yapılandırılmış LLM değerlendirmesi sonucunda Claude en dengeli çıktı üretmiş, Gemini benzer şekilde güçlü bir analiz sunmuş, DeepSeek ise trendleri doğru yorumlamasına rağmen DAM metriğini hatalı ele aldığı için daha düşük değerlendirilmiştir. Bu sonuç, LLM çıktılarının mutlaka metrik bilgisi ve alan bilgisiyle eleştirel biçimde denetlenmesi gerektiğini göstermektedir.

Ek A. LLM Promptları ve Değerlendirme Rubriği

Aşağıdaki promptlar ChatGPT, Gemini, Claude veya DeepSeek gibi modellere aynı formatta verilerek modellerin karşılaştırılabilir çıktı üretmesi sağlanabilir.

A.1. Tek Sürüm Değerlendirme Promptu

Aşağıda picocli projesinin bir sürümüne ait CK metrikleri ve QMOOD skorları verilmiştir. Görevin: (1) genel tasarım kalitesini değerlendir, (2) bakım yapılabilirlik risklerini belirt, (3) coupling, cohesion, complexity ve inheritance açısından yorum yap, (4) teknik borç belirtilerini açıkla, (5) somut refactoring önerileri ver. Sadece verilen metriklere dayan; verilmeyen sınıf/kod ayrıntılarını uydurma.

A.2. Sürüm Karşılaştırma Promptu

Aşağıda picocli projesinin iki sürümüne ait metrikler verilmiştir. Görevin: kalite değişimini açıkla, iyileşen ve kötüleşen kalite niteliklerini belirt, mimari bozulma işaretlerini değerlendir, teknik borç artışı/azalışı hakkında yorum yap ve refactoring öncelikleri öner. Yorumlarını metrik değerlerine dayandır.

A.3. 10 Sürüm Trend Analizi Promptu

Aşağıda picocli projesinin 10 sürümlük QMOOD kalite skorları ve temel metrik trendleri verilmiştir. Görevin: yazılımın kalite evrimini dönemlere ayır, kritik kalite düşüşlerini belirle, TQI trendini yorumla, understandability, reusability, flexibility ve extendibility açısından genel eğilimi açıkla, teknik borç birikimi olup olmadığını değerlendir ve gelecek sürümler için mimari iyileştirme önerileri ver.

A.4. LLM Değerlendirme Rubriği

Tablo 11. LLM değerlendirme rubriği

Kriter	Açıklama	Puan
Metriklerle bağlılık	Yanıt verilen sayısal verilere dayanıyor mu?	1-5
Tutarlılık	Yanıt kendi içinde çelişiyor mu?	1-5
Teknik doğruluk	OO/QMOOD kavramları doğru kullanılıyor mu?	1-5
Refactoring kalitesi	Öneriler somut ve uygulanabilir mi?	1-5
Halüsinasyon riski	Verilmeyen bilgi uyduruluyor mu?	1-5
Sürüm farkını anlama	Trend ve sürüm farkları doğru yorumlanıyor mu?	1-5

Kaynakça

- Bansiya, J., & Davis, C. G. (2002). A hierarchical model for object-oriented design quality assessment. IEEE Transactions on Software Engineering, 28(1), 4-17.
- Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. IEEE Transactions on Software Engineering, 20(6), 476-493.
- McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, 2(4), 308-320.
- Buzluca, F. (2012-2025). Yazılım Tasarımı Kalitesi - Yazılım Metrikleri (ytk04_2.pdf). Ders notları.
- Buzluca, F. (2012-2025). Yazılım Tasarımı Kalitesi - QMOOD Kalite Modeli (ytk05_2.pdf). Ders notları.
- Yazılım Mimarileri ve Tasarım Yöntemleri Dersi. QMOOD Tabanlı Yazılım Kalitesi Analizi ve LLM Destekli Değerlendirme Proje Yönergesi.
- CK Tool. <https://github.com/mauricioaniche/ck>
- picocli. <https://github.com/remkop/picocli>